
Neural Networks For NLP

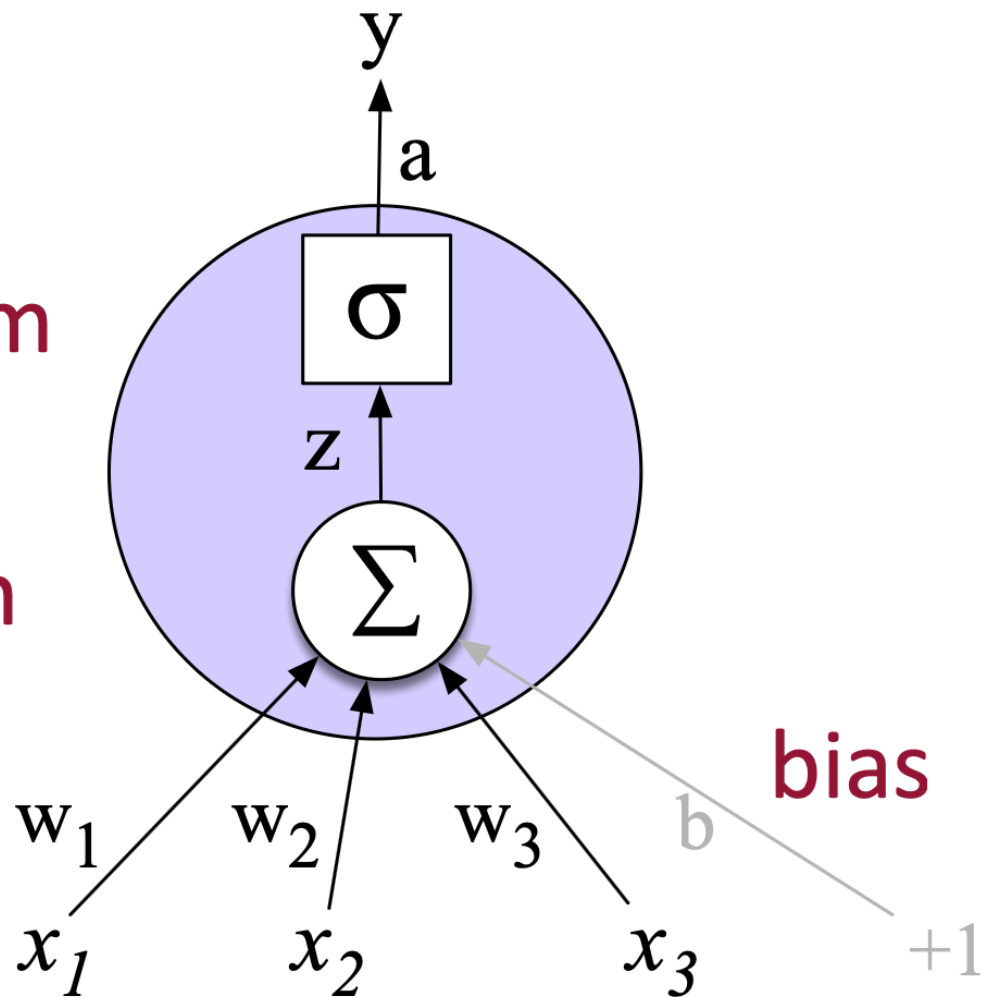
Output value

Non-linear transform

Weighted sum

Weights

Input layer



Take weighted sum of inputs, plus a bias

$$z = b + \sum_i w_i x_i$$

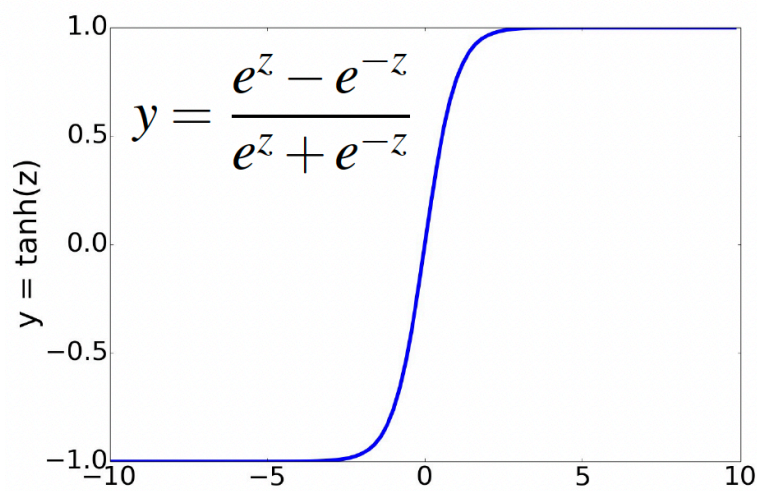
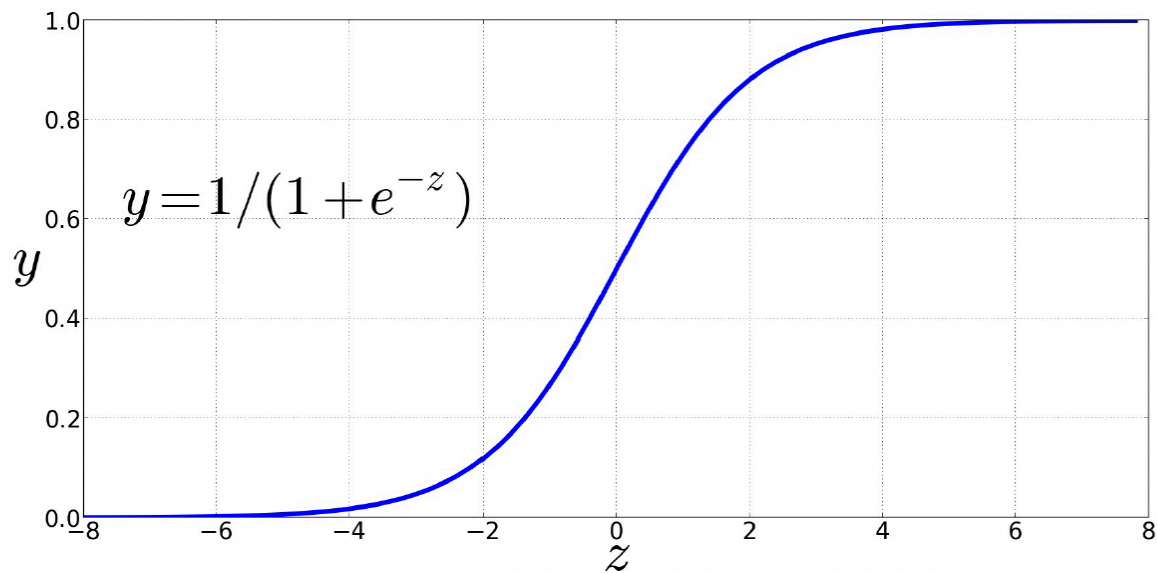
$$z = w \cdot x + b$$

Instead of just using z , we'll apply a nonlinear activation function f :

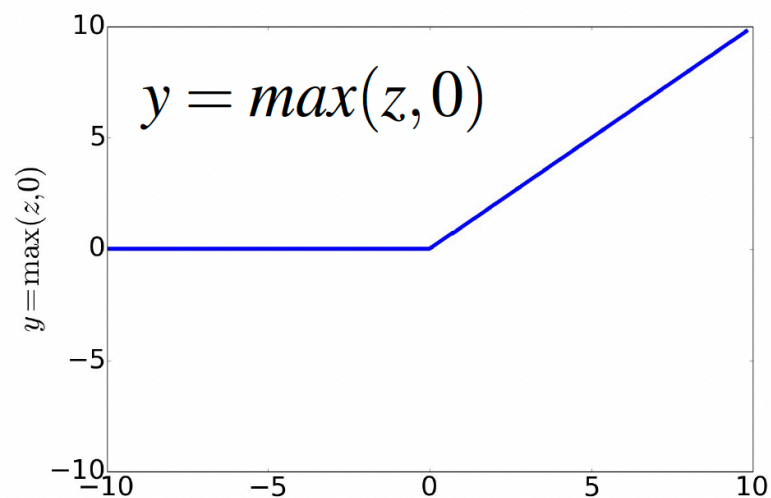
$$y = a = f(z)$$

Sigmoid

$$y = \sigma(z) = \frac{1}{1 + e^{-z}}$$



tanh



ReLU

Rectified Linear Unit

Two-Layer Network with scalar output

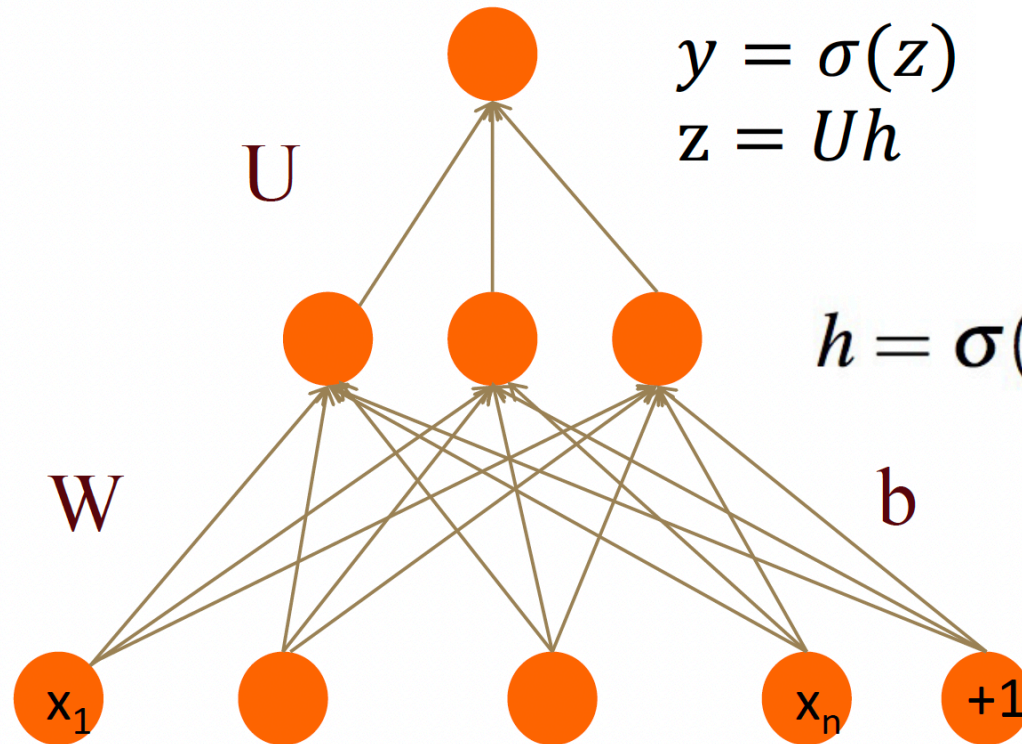
Output layer
(σ node)

$$y = \sigma(z)$$
$$z = Uh$$

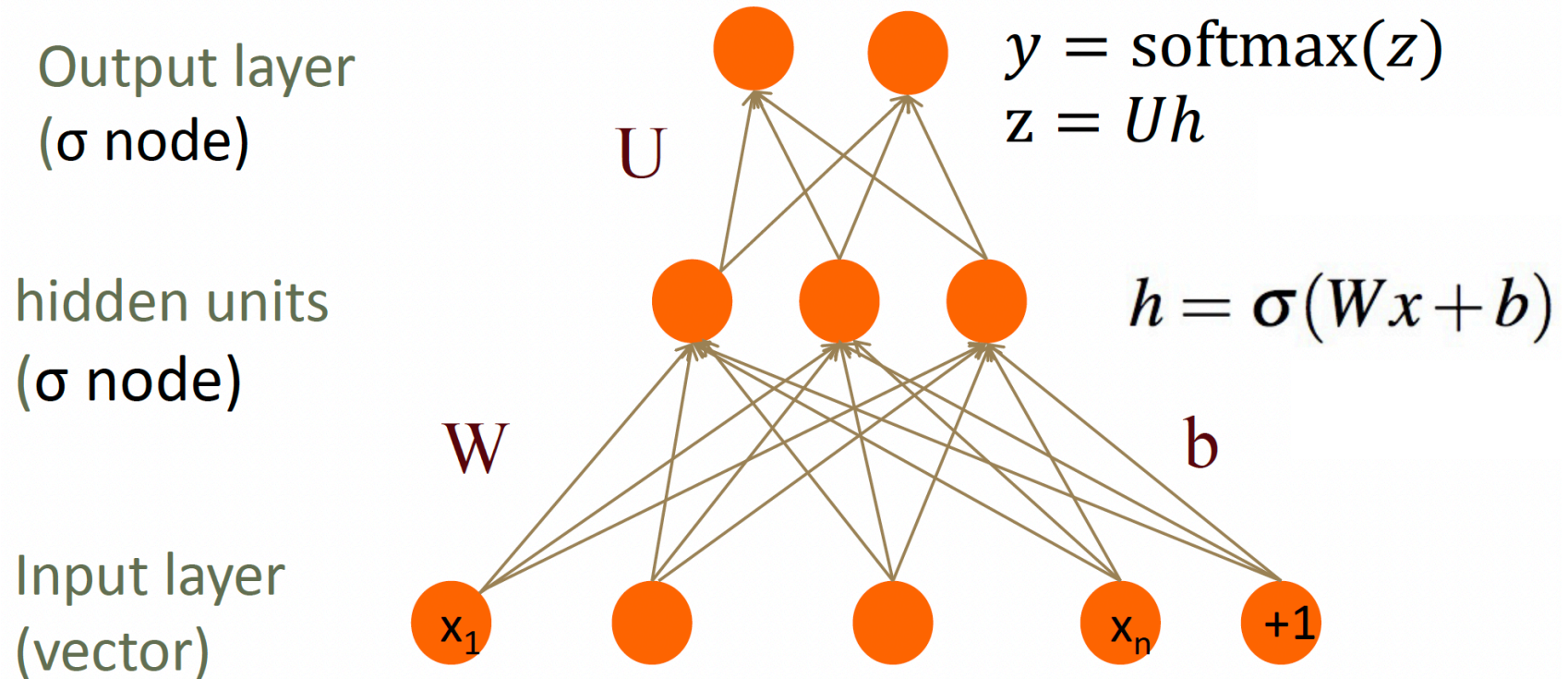
hidden units
(σ node)

$$h = \sigma(Wx + b)$$

Input layer
(vector)



Two-Layer Network with softmax output

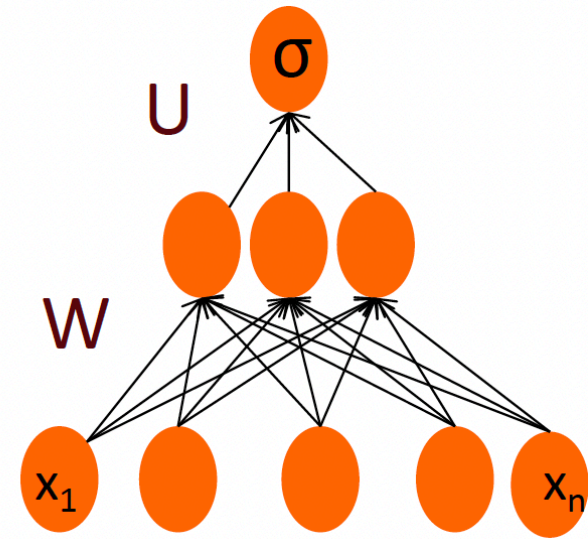


Classification: Sentiment Analysis

We could do exactly what we did with logistic regression

Input layer are binary features as before

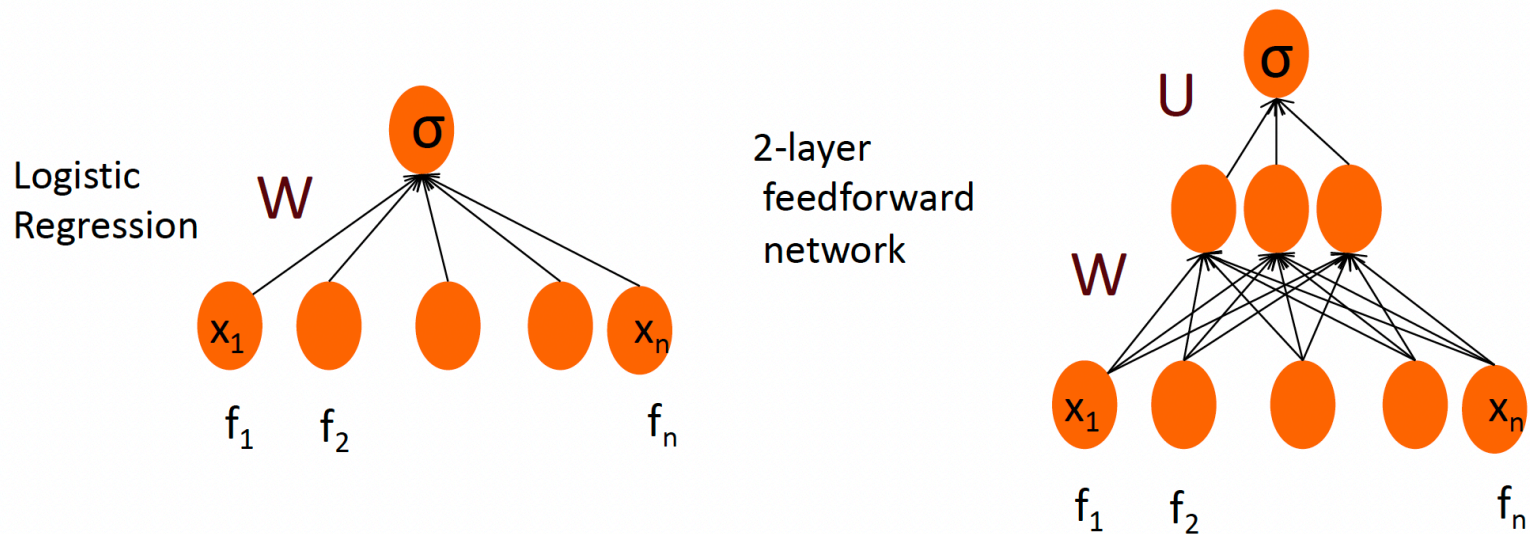
Output layer is 0 or 1



Sentiment Features

Var	Definition
x_1	$\text{count}(\text{positive lexicon}) \in \text{doc}$
x_2	$\text{count}(\text{negative lexicon}) \in \text{doc}$
x_3	$\begin{cases} 1 & \text{if "no"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$
x_4	$\text{count}(\text{1st and 2nd pronouns}) \in \text{doc}$
x_5	$\begin{cases} 1 & \text{if "!"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$
x_6	$\log(\text{word count of doc})$

Feedforward nets for simple classification



Just adding a hidden layer to logistic regression

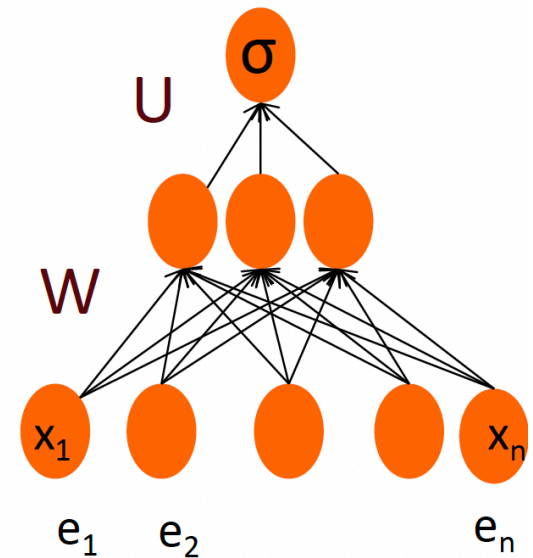
- allows the network to use non-linear interactions between features
- which may (or may not) improve performance.

Even better: representation learning

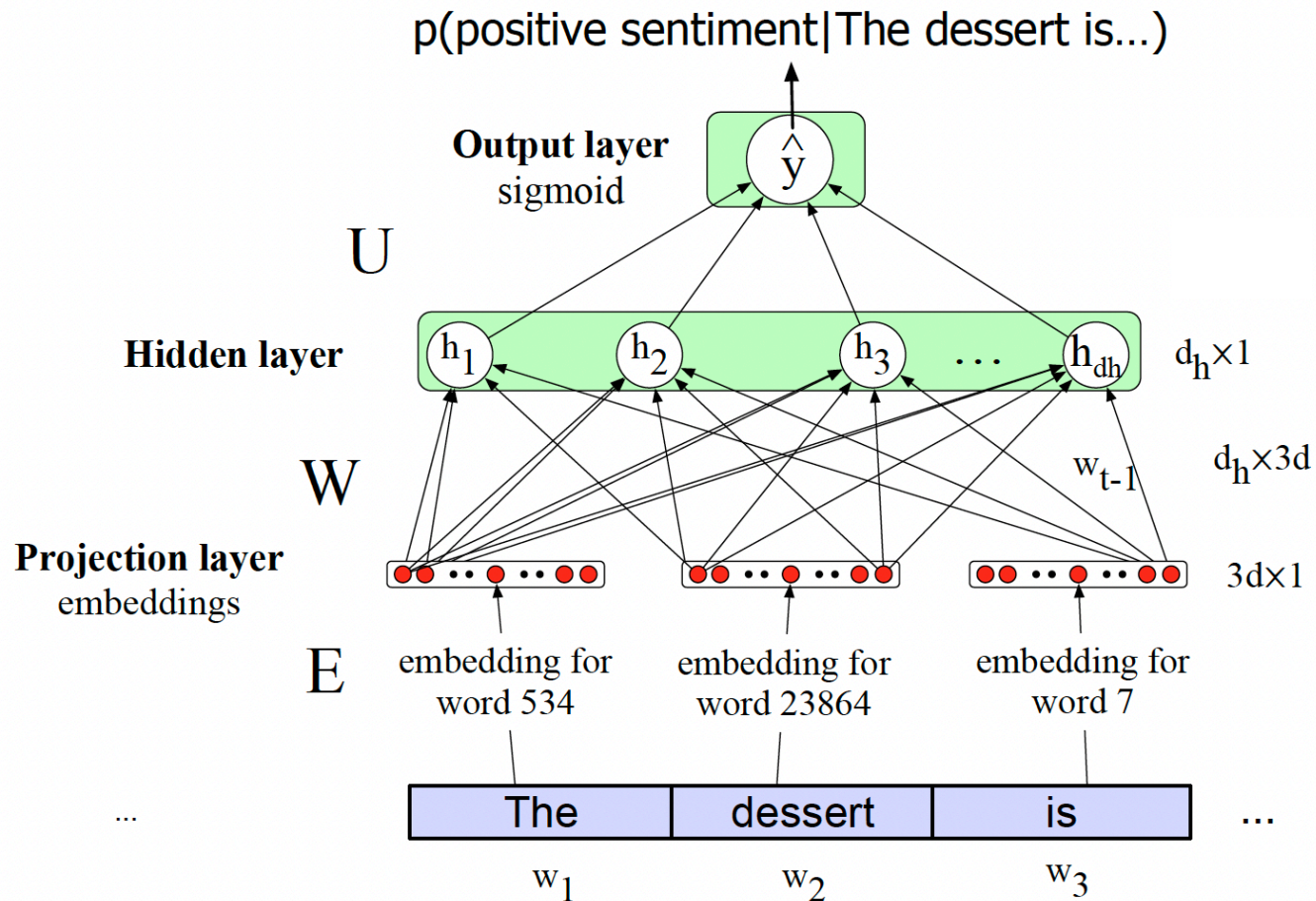
The real power of deep learning comes from the ability to **learn** features from the data

Instead of using hand-built human-engineered features for classification

Use learned representations like embeddings!



Neural Net Classification with embeddings as input features!



Simple feedforward Neural Language Models

Task: predict next word w_t

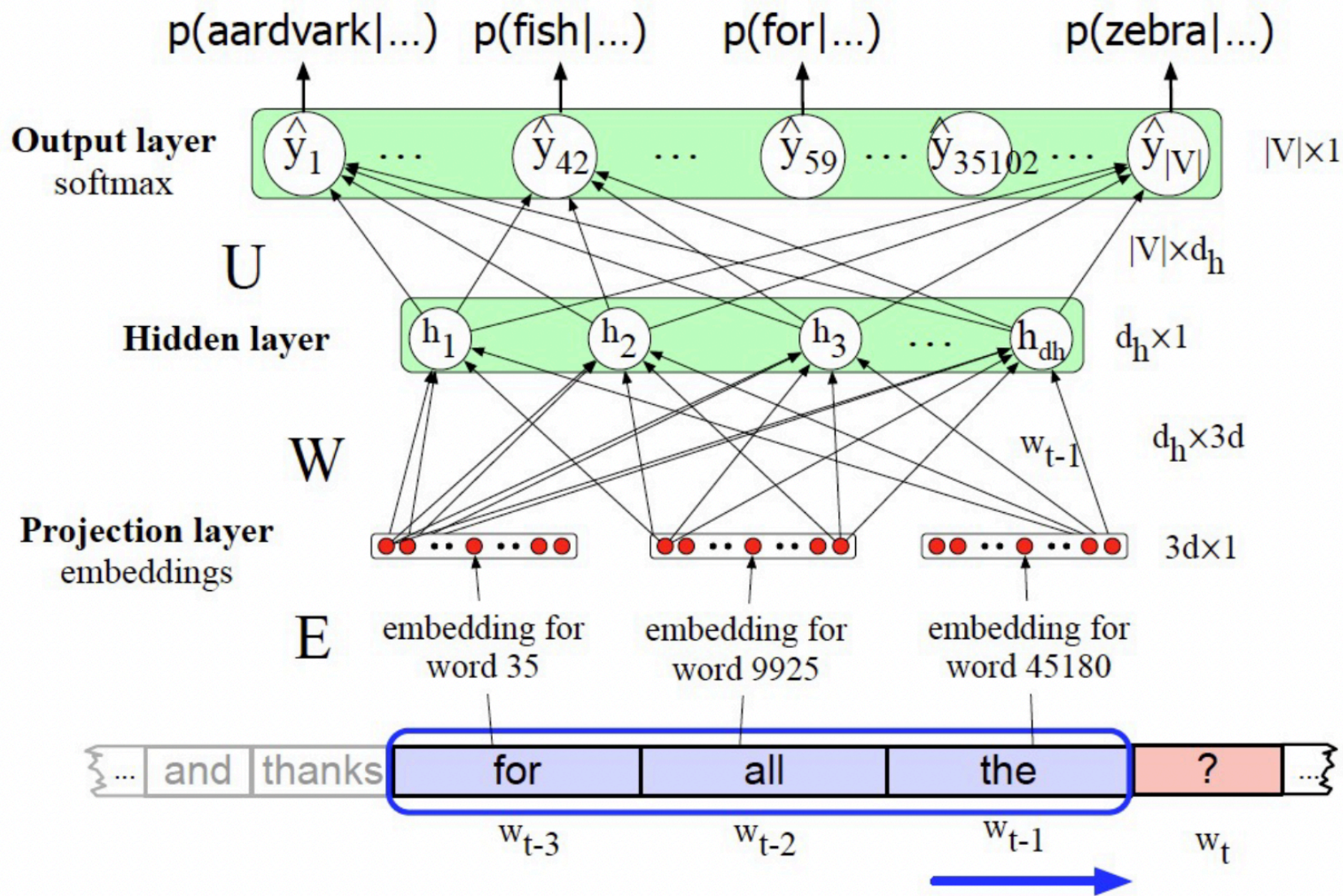
given prior words $w_{t-1}, w_{t-2}, w_{t-3}, \dots$

Problem: Now we're dealing with sequences of arbitrary length.

Solution: Sliding windows (of fixed length)

$$P(w_t | w_1^{t-1}) \approx P(w_t | w_{t-N+1}^{t-1})$$

Neural Language Model



Why Neural LMs work better than N-gram LMs

Training data:

We've seen: I have to make sure that the cat gets fed.

Never seen: dog gets fed

Test data:

I forgot to make sure that the dog gets ____

N-gram LM can't predict "fed"!

Neural LM can use similarity of "cat" and "dog" embeddings to generalize and predict "fed" after dog

For training, we need the derivative of the loss with respect to weights in early layers of the network

- But loss is computed only at the very end of the network!

Solution: **backward differentiation**

Modeling Time in Neural Networks

Language is inherently temporal

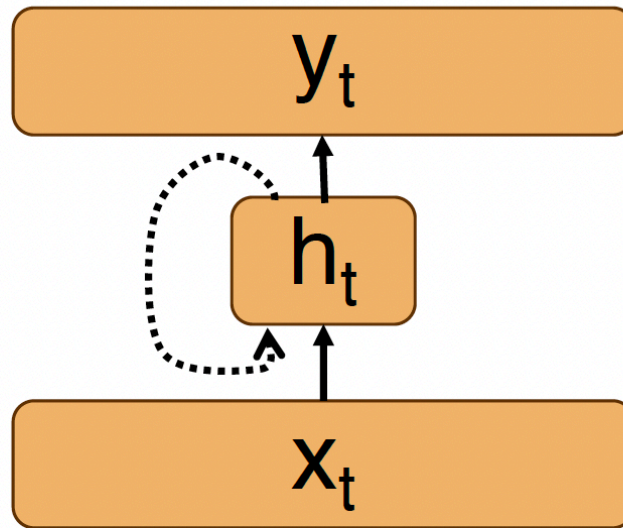
Yet the simple NLP classifiers we've seen (for example for sentiment analysis) mostly ignore time

- (Feedforward neural LMs (and the transformers we'll see later) use a "moving window" approach to time.)

Here we introduce a deep learning architecture with a different way of representing time

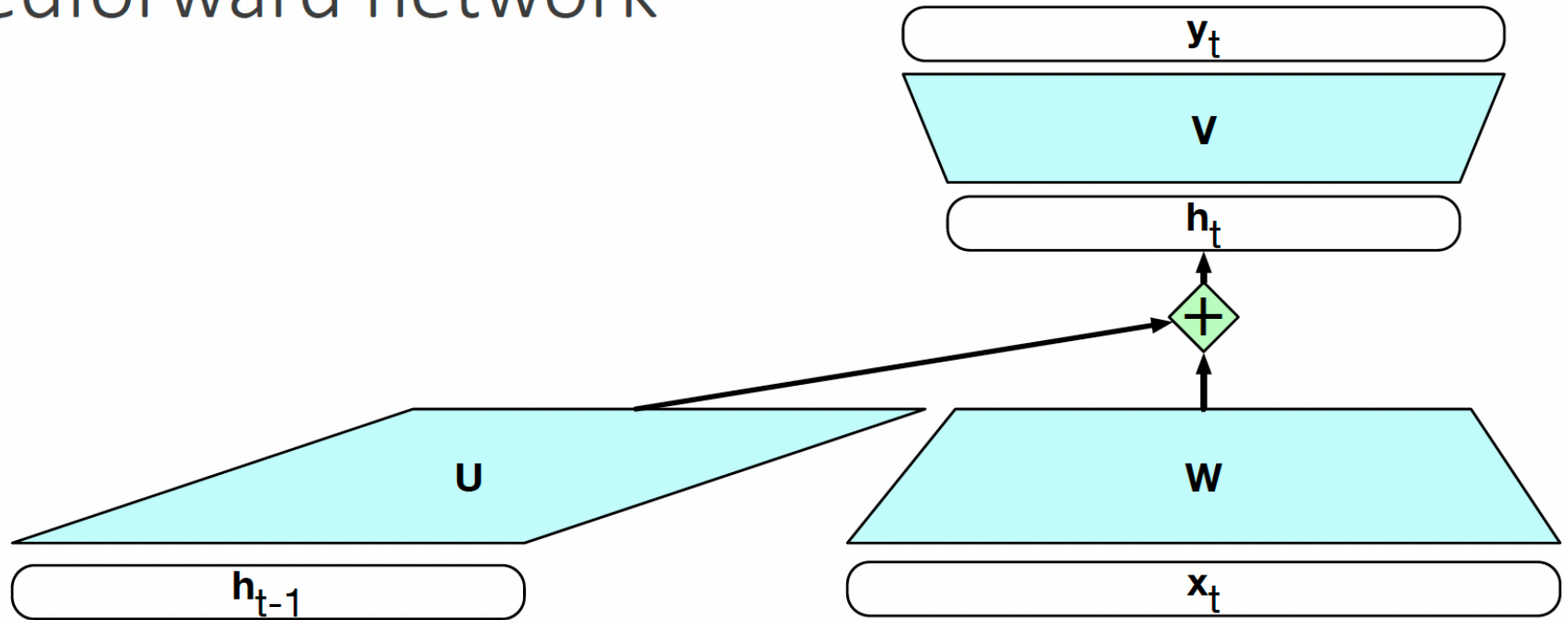
- RNNs and their variants like LSTMs

Simple Recurrent Nets



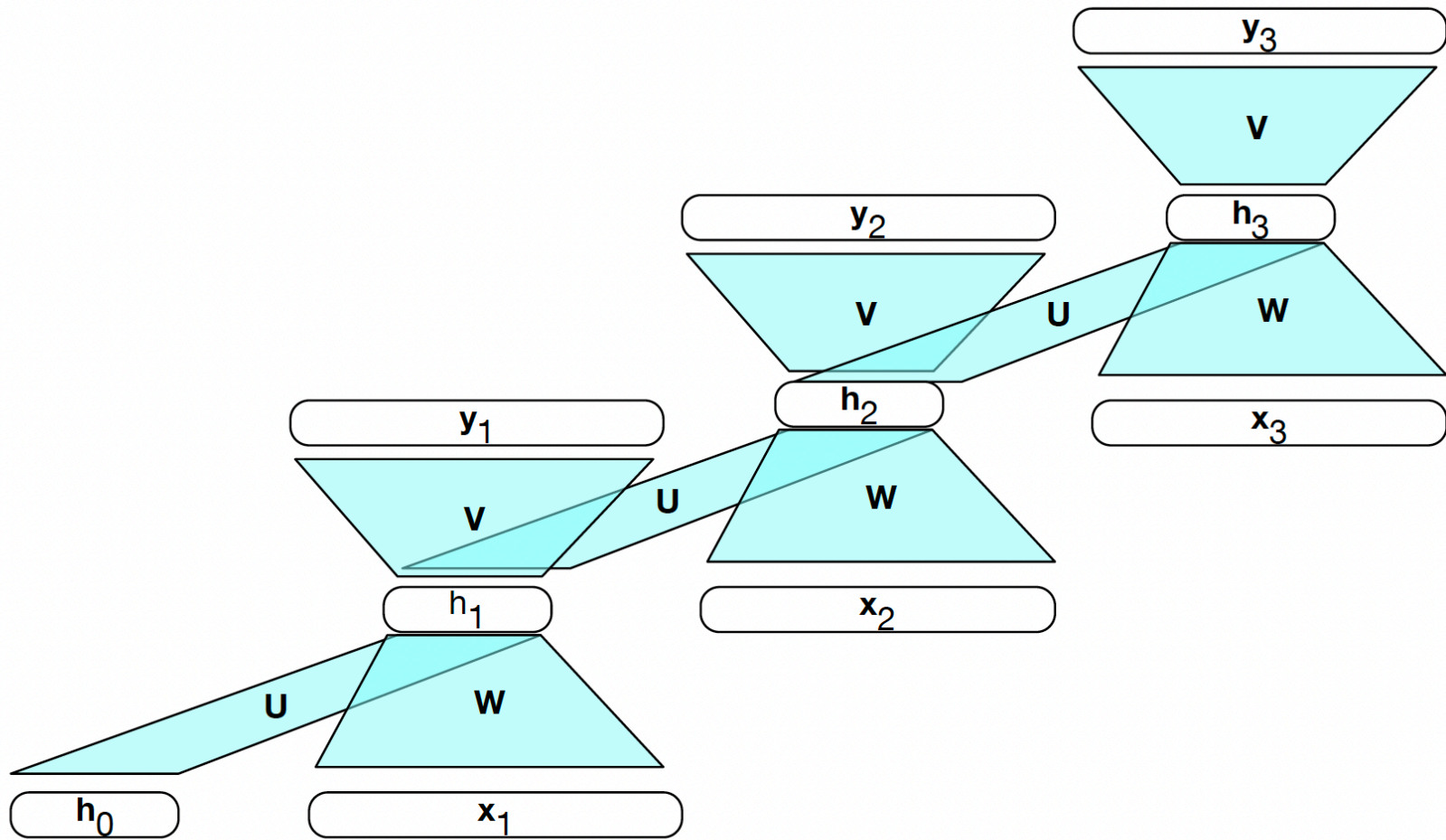
The hidden layer has a recurrence as part of its input
The activation value h_t depends on x_t but also h_{t-1} !

Simple recurrent neural network illustrated as a feedforward network



$$\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t)$$

$$\mathbf{y}_t = \text{softmax}(\mathbf{V}\mathbf{h}_t)$$



Reminder: Language Modeling

$$P(\textit{fish} | \textit{Thanks for all the})$$

$$P(w_{1:n}) = \prod_{i=1}^n P(w_i | w_{<i})$$

The size of the conditioning context for different LMs

The n-gram LM:

Context size is the $n - 1$ prior words we condition on.

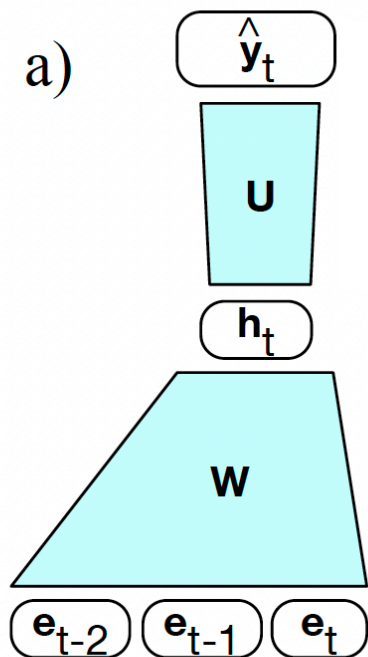
The feedforward LM:

Context is the window size.

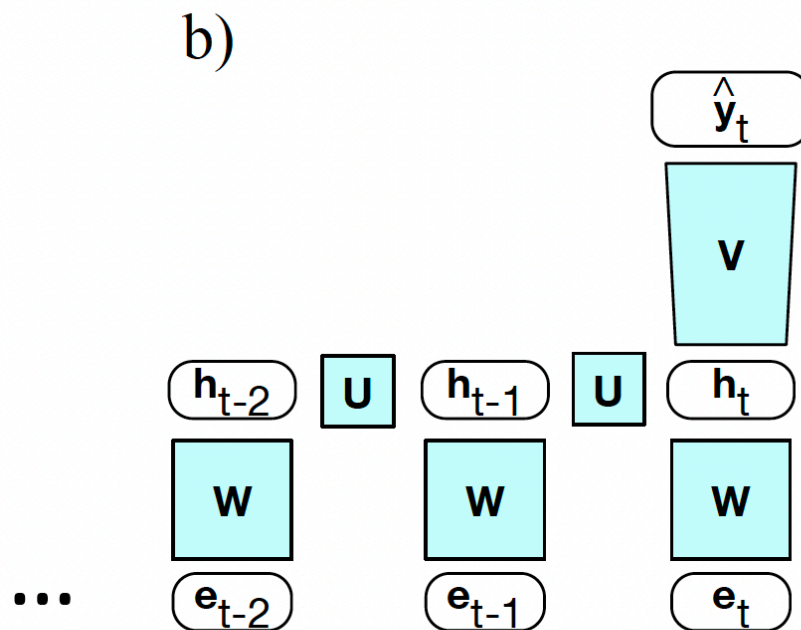
The RNN LM:

No fixed context size; h_{t-1} represents entire history

FFN LMs vs RNN LMs



FNN



RNN